Date: 17/03/2026
Name: Dhanya S

I am Dhanya S from Kakunje Software working as an Intern. Today I had my 38 session it was an interesting session and mam had taught us about MongoDB and NodeJS.
Today i learned about the mongodb queries and how to insert it in the mongodb shell, and also i had learned about how to add the CRUD operations in node.js and sending it to the postman and storing that database in mongodb.

config > JS db.js > default

```
1   import mongoose from "mongoose";
2
3   const connectDB = async ()=>{
4       try{
5           await mongoose.connect(process.env.MONGO_URI);
6           console.log("mongodb connected");
7       }catch(error){
8           console.log("database connection failed",error);
9           process.exit(1);
10      }
11  };
12  export default connectDB;
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\Dhanya\OneDrive\Desktop\mongodb> node server.js
[dotenv@17.3.1] injecting env (2) from .env -- tip: 🛠 run anywhere with `dotenvx run -- yourcomm
server is running on port 5000
mongodb connected
```

models > JS Product.js > default

```
1   import mongoose from "mongoose";
2
3   const productSchema = new mongoose.Schema({
4       name:{
5           type:String,
6           required:true
7       },
8       price:{
9           type:Number
10      },
11      category:{
12          type:String,
13          required:true
14      },
15      stock:{
16          type:Number
17      }
18  },{timestamps:true});
19  const Product = mongoose.model("Product",productSchema);
20  export default Product;
21
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\Dhanya\OneDrive\Desktop\mongodb> node server.js
[dotenv@17.3.1] injecting env (2) from .env -- tip: 🛠 run anywhere with `dotenvx run -- your
server is running on port 5000
mongodb connected
```

routes > JS productroutes.js > ...

```
1   import express from "express";
2   import Product from "../models/Product.js";
3
4   const router = express.Router();
5   router.post("/products",async(req,res)=>{
6       try{
7           const product = new Product(req.body);
8           await product.save();
9           res.status(201).json(product);
10      }catch(error){
11          res.status(500).json({message:error.message});
12      }
13  })
14
15  router.get("/products",async(req,res)=>{
16      try{
17          const products = await Product.find();
18          res.json(products);
19      }catch(error){
20          res.status(500).json({message:error.message});
21      }
22  })
23
24  router.get("/products/:id",async(req,res)=>{
25      try{
26          const product = await Product.findById(req.params.id);
27          if(!product){
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\Dhanya\OneDrive\Desktop\mongodb> node server.js
[dotenv@17.3.1] injecting env (2) from .env -- tip: 🛠 run anywhere with `dotenvx run -- yourcomma
server is running on port 5000
mongodb connected
```

routes > JS productroutes.js > ...

```
24  router.get("/products/:id",async(req,res)=>{
26          const product = await Product.findById(req.params.id);
27          if(!product){
28              return res.status(404).json({message:"product not found"});
29          }
30          res.json(product);
31      }catch(error){
32          res.status(500).json({message:error.message});
33      }
34  });
35
36  router.put("/products/:id",async (req,res)=>{
37      try{
38          const updatedProduct=await Product.findByIdAndUpdate(
39              req.params.id,
40              req.body,
41              {new:true}
42          );
43          if(!updatedProduct){
44              return res.status(404).json({message:"product not found"});
45          }
46          res.json(updatedProduct);
47      }catch(error){
48          res.status(500).json({message:error.message});
49      }
50  });
51
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\Dhanya\OneDrive\Desktop\mongodb> node server.js
[dotenv@17.3.1] injecting env (2) from .env -- tip: 🛠 run anywhere with `dotenvx run -- yourcommand`
server is running on port 5000
mongodb connected
```

```
        );
        if(!updatedProduct){
            return res.status(404).json({message:"product not found"});
        }
        res.json(updatedProduct);
    }catch(error){
        res.status(500).json({message:error.message});
    }
});

router.delete("/products/:id",async(req,res)=>{
    try{
        const deletedProduct=await Product.findByIdAndDelete(req.params.id);
        if(!deletedProduct){
            return res.status(404).json({message:"product not found"});
        }
        res.json({message:"product deleted successfully"});
    }catch(error){
        res.status(500).json({message:error.message})
    }
});
export default router;

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\Dhanya\OneDrive\Desktop\mongodb> node server.js
[dotenv@17.3.1] injecting env (2) from .env -- tip: 🛠 run anywhere with `dotenvx run -- yourcomman
server is running on port 5000
mongodb connected
```

.env

Import to Postman

```
# PORT = 5000
# MONGO_URI = mongodb://localhost:27017/users

PORT = 5000
MONGO_URI = mongodb://localhost:27017/products
```

JS server.js > ...

```
17  //products
18  import express from "express";
19  import dotenv from "dotenv";
20  import connectDB from "./config/db.js";
21  import productRoutes from "./routes/productroutes.js";
22
23  dotenv.config();
24  connectDB();
25  const app = express();
26  app.use(express.json());
27  app.use("/api",productRoutes);
28  const PORT = process.env.PORT;
29  app.listen(PORT,()=>{
30      console.log(`server is running on port ${PORT}`)
31  })
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\Dhanya\OneDrive\Desktop\mongodb> node server.js
[dotenv@17.3.1] injecting env (2) from .env -- tip: 🛠 run anywhere with `dotenvx run -- yo
server is running on port 5000
mongodb connected
```

http://localhost:5000/api/products

POST http://localhost:5000/api/products

Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
{
"name":"Laptop",
"price":50000,
"category":"Electronics",
"stock":10
}
```

Body Cookies Headers (7) Test Results

Pretty Raw Preview JSON


```
{
    "name": "Laptop",
    "price": 50000,
    "category": "Electronics",
    "stock": 10,
    "_id": "69b949688eed3c1f491dd793",
    "createdAt": "2026-03-17T12:30:32.631Z",
    "updatedAt": "2026-03-17T12:30:32.631Z",
    "__v": 0
}
```

http://localhost:5000/api/products

GET http://localhost:5000/api/products

Params Authorization Headers (7) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL

This request does no

Body Cookies Headers (7) Test Results

Pretty Raw Preview JSON


```
[
    {
        "_id": "69b949688eed3c1f491dd793",
        "name": "Laptop",
        "price": 50000,
        "category": "Electronics",
        "stock": 10,
        "createdAt": "2026-03-17T12:30:32.631Z",
        "updatedAt": "2026-03-17T12:30:32.631Z",
        "__v": 0
    }
]
```

products +

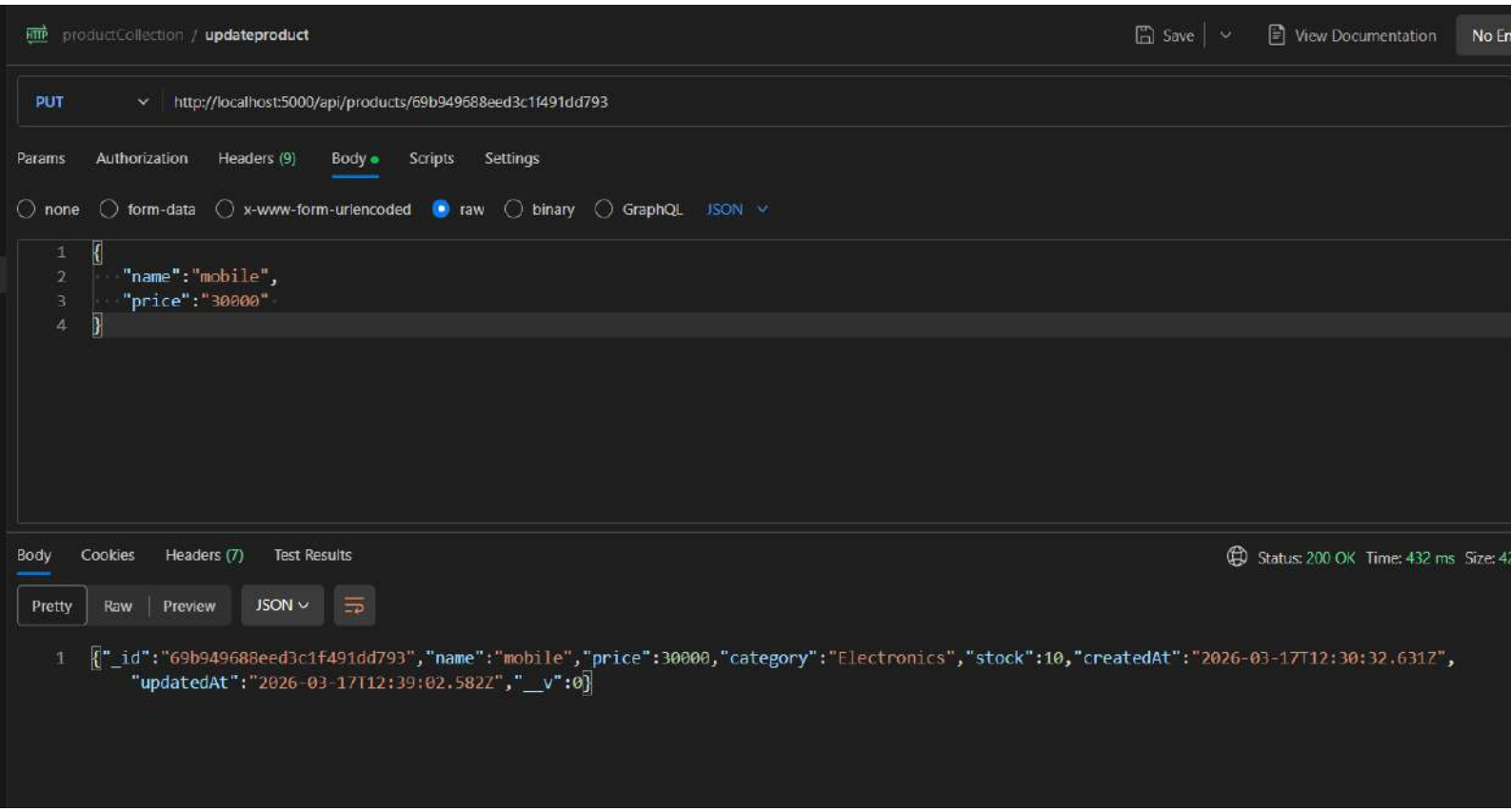
products > products > products

Documents 1 | Aggregations | Schema | Indexes 1 | Validation

Type a query: { field: 'value' } or Generate query

ADD DATA | UPDATE | DELETE | EXPORT DATA | EXPORT CODE


```
_id: ObjectId('69b949688eed3c1f491dd793')
name : "Laptop"
price : 50000
category : "Electronics"
stock : 10
createdAt : 2026-03-17T12:30:32.631+00:00
updatedAt : 2026-03-17T12:30:32.631+00:00
__v : 0
```


productCollection / updateproduct
Save
View Documentation
PUT
http://localhost:5000/api/products/69b949688eed3c1f491dd793
Params
Authorization
Headers (9)
Body
Scripts
Settings
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
{
"name":"mobile",
"price":"30000"
}
Body
Cookies
Headers (7)
Test Results
Status: 200 OK
Time: 432 ms
Pretty
Raw
Preview
JSON
{"_id":"69b949688eed3c1f491dd793","name":"mobile","price":30000,"category":"Electronics","stock":10,"createdAt":"2026-03-17T12:30:32.631Z",
"updatedAt":"2026-03-17T12:39:02.582Z","__v":0}

products +

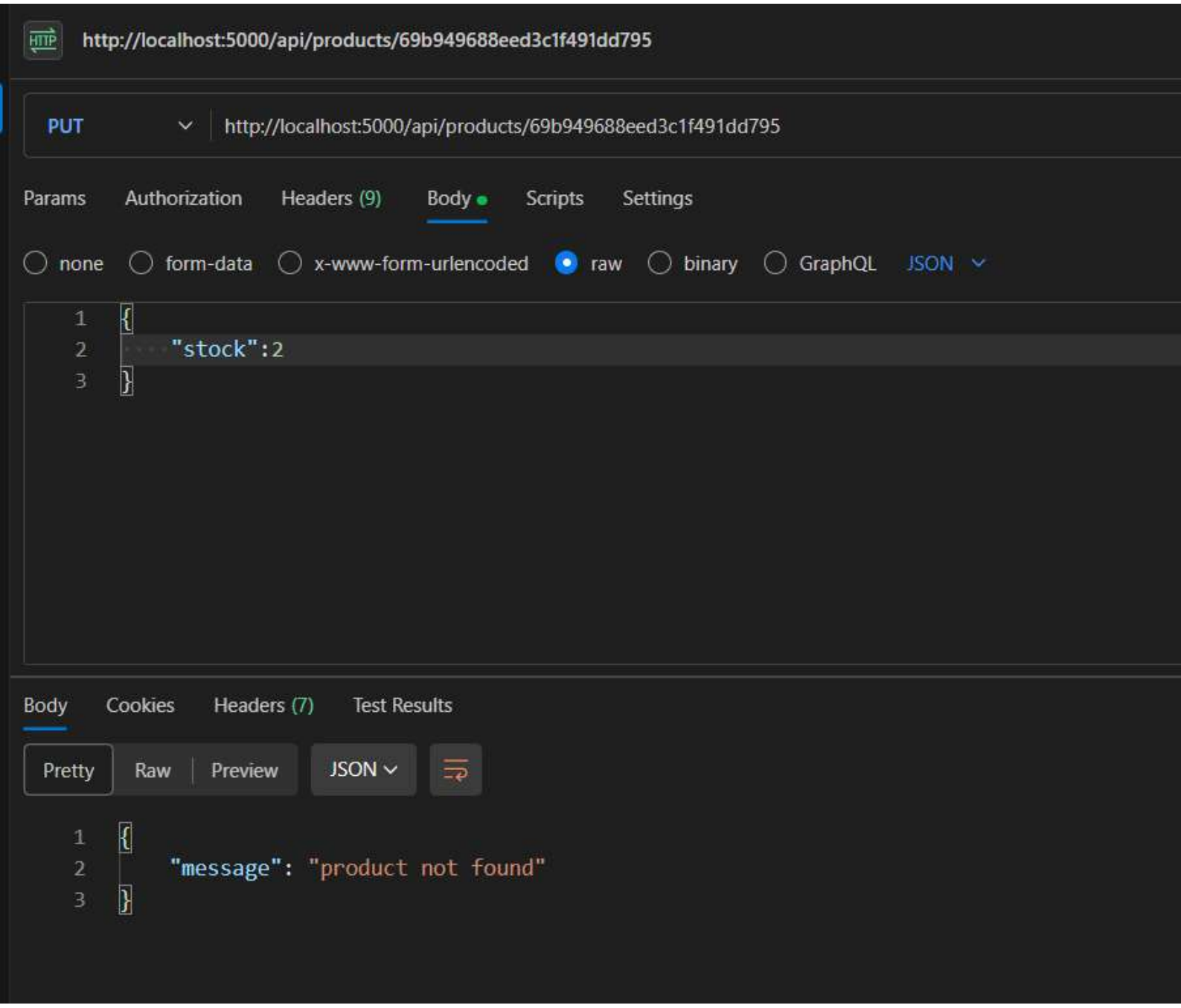
products > products > products

Documents 1 | Aggregations | Schema | Indexes 1 | Validation

Type a query: { field: 'value' } or Generate query

ADD DATA | UPDATE | DELETE | EXPORT DATA | EXPORT CODE


```
_id: ObjectId('69b949688eed3c1f491dd793')
name : "mobile"
price : 30000
category : "Electronics"
stock : 10
createdAt : 2026-03-17T12:30:32.631+00:00
updatedAt : 2026-03-17T12:39:02.582+00:00
__v : 0
```


http://localhost:5000/api/products/69b949688eed3c1f491dd795
PUT
http://localhost:5000/api/products/69b949688eed3c1f491dd795
Params
Authorization
Headers (9)
Body
Scripts
Settings
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
1 {
2 "stock":2
3 }
Body
Cookies
Headers (7)
Test Results
Pretty
Raw
Preview
JSON
1 {
2 "message": "product not found"
3 }

productCollection / **deleteproduct**

DELETE | http://localhost:5000/api/products/69b949688eed3c1f491dd793

Params | Authorization | Headers (7) | Body | Scripts | Settings

Query Params

	Key	Value
	Key	Value

Body | Cookies | Headers (7) | Test Results

Pretty | Raw | Preview | JSON


```
{
    "message": "product deleted successfully"
}
```